

Class-4

In: list = [1, 2, 3, 4, 5]

list[2]

Out: 3

In: list[1:4]

Out: [2, 3, 4]

index: 1 4-1

In: list.append(8)

list

Out: [1, 2, 3, 4, 5, 8]

In: list.insert(1, 'a')

list

index at which element to be added.

Element to be added.

Out: [1, 'a', 2, 3, 4, 5, 8]

In: list.extend([9, 10])

list

Out: [1, 'a', 2, 3, 4, 5, 8, 9, 10]

In: list.remove('a')

list

Out: [1, 2, 3, 4, 5, 8, 9, 10]

In: l2 = [1, 2, 3, 2, 4]

l2.remove(2)

l2

Out: [1, 3, 2, 4]

↑ Removes the first element "2" in the list l2.

In: list.pop()

Out: 10

No argument

↑ Pops out last element of the list.

In: list.pop(2)

Out: 3

↑ Pops out the element at index-2

In: list.clear()

↳ Clears entire list.

□ A dictionary in python is a collection of items where each item consists of a key and a value. Dictionaries are optimized for retrieving data when keys are known. Unlike lists which are indexed by a range of nos, dictionaries are indexed by keys which can be of any type. Characteristics of dictionary:

- ① Unordered
- ② Mutable
- ③ Indexed by keys.

*** "{ }" are used to declare a dictionary.

In: d = { 'name': 'abc', 'age': 25 }

d key value key value.

type(d)

Out: { 'name': 'abc', 'age': 25 }

dict.

In: d['add'] = 'xyz'

↑ key ↑ value

d['age'] = 26

Out: { 'name': 'abc', 'age': 26, 'add': 'xyz' }

In: d['name'] → 'abc'

In: d.keys() → prints all keys

d.values() → prints all values.

d.items() → prints all items, like ('name', 'abc') etc.

In: d.pop('age') → pops the item ('age', 26).

d

Out: 26

{ 'name': 'abc', 'add': 'xyz' }

In: d.clear()

d

Out: { }

□ A tuple is an ordered immutable collection of elements in python. Unlike lists, once a tuple is created, its elements can't be modified. ~~But~~ Tuples are often used to group related data together, and can contain elements of different datatypes.

In : tup = (1, 2, 3)
 tup1 = (4, 5, 6)
 tup + tup1
 Out : (1, 2, 3, 4, 5, 6)
 In : tup * 3
 Out : (1, 2, 3, 1, 2, 3, 1, 2, 3)

In : 4 in tup
 Out : False
 In : 4 in tup1
 Out : True
 In : tup[1]
 Out : 2 ↑ index

In : tup2 = (1, 2, 3, 3, 4, 4, 3, 7, 8, 7, 8)
 tup2.count(3)
 Counts no. of times "3" appears in tup2.
 Out : 3
 In : tup2.index(3)
 Out : 2. returns index of the first occurrence of "3" in the tuple.

□ A set in python is an unordered collection of unique elements. Sets are commonly used for membership testing, removing duplicates from a sequence, and mathematical operations like union, intersection, and difference.

In : set = {1, 2, 3, 4, 5, 4, 5, 2}
 set
 Out : {1, 2, 3, 4, 5}
 In : set.add(8)
 set
 Out : {1, 2, 3, 4, 5, 8}

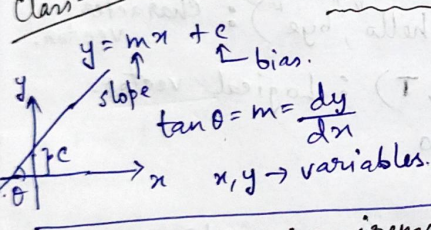
In : set.update({9, 10})
 set
 Out : {1, 2, 3, 4, 5, 8, 9, 10}

In : set.remove(9) → Removes element "9"
 In : set.pop() → Pops 1st element of the set.
 Out : 1

* set.clear() → clears the set

Class-5

INTRODUCTION TO R



requirements.txt

```

pandas = 1.2
numpy = 1.3
...

```

In Indian car market,
 bias → mileage
 → price.

Python :
 pip install pandas
 pip install numpy
 pip install matplotlib
 import pandas as pd
 pd.read() Jupyter
 pip install -r requirements.txt
 ↑
 read

- Install R : <https://cran.rstudio.com/>
- Install RStudio : <https://www.rstudio.com/products/rstudio/download/>
- Install swirl and learnr packages. Type in RStudio terminal :

```

> install.packages("swirl")
> install.packages("learnr")
> library("swirl")
> swirl()

```

 } → Next, proceed as necessary.
 Choose the options (e.g. 1, 2, 3, ...) as applicable.
- * "%" at the right side shows how much you have completed, e.g. 0%, 5%, 8% etc.
- * If "..." comes up, keep pressing Enter for the next part.

Class-6
 Python

install python
 { install jupyter lab
 spyder

R

install R
 install RStudio

These are IDEs

(Integrated Development Environment)

python

• ipynb • py

• ipynb : interactive notebook

• py : just python script.

Introduction to Programming and Tools - induction

class-1, 2 → see word file.

class 2 (Cont.)

String: collection of characters (alpha-numeric, special characters)

In: `x = "abc&*#45h"`
type(x)

Out: str

In: `x = "abc" + "1111" + "1111"`

type(x)

Out: str

a	b	c	d	e	f	g	h	:	string
0	1	2	3	4	5	6	7	→	Indexing
-8	-7	-6	-5	-4	-3	-2	-1	→	Reverse indexing.

In: `x = "abcdefgh"`
x[2]
Out: 'c'

In: `x[-3]`
Out: 'f'

* Comments in Python: `#` (single line)

or, `"""` (multiple line)

Class-3.

In: `x = None`
type(x)

Output: NoneType

In: `x, y, z = 4, 5, 6` → multiple variables defined
`del x` → Deletes variable 'x'.

In: `import keyword`
keyword.kwlist.

Out: all keywords printed

Can't use these keywords to define variables.

In: `a = 5`
`b = 10`

Out: Indentation error

■ A string is a sequence of characters enclosed within single quotes, double quotes, triple quotes. Strings are used to represent text in python.

* Concatenation of strings:

In: `str = 'Hello'`
`str1 = 'World'`

`str + str1` → 'HelloWorld'

`str + ' ' + str1` → 'Hello World'.

In: `str = 'bigdataanalysis'`
`str[0:5]`

Out: 'bigda'

In: `str[-16:-11]`

Out: 'bigd'

In: `str[-14:]`

Out: 'gdataanalysis'

In: `str[0:9:3]`

Out: 'bda'

Repetition of strings:

→ `str2 = 'ha'`

`str2 * 3` → 'hahaha'.

In: `str = 'big'`

`str.upper()`

Out: 'BIG'

In: `str = 'BIG'`

`str.lower()`

Out: 'big'

In: `str = 'bigdata'`

`str.capitalize()`

Out: 'Bigdata'

In: `f = 'helloworld'`
`f.replace('world', 'india')`

Out: 'helloindia'

In: `f = 'qwerty'`

`f(4)`

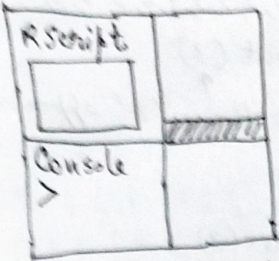
Out: 't'

■ A list in python is an ordered collection of items. Lists are datatypes that contain different types of elements (int, float, str). Lists are mutable, meaning the elements within a list can be changed after the list has been created.

In: `list = [1, 2, 3, 4, 3.4, 'apple']`

list

Out: prints the list.



Indexing:

R: 1 2 3 4 ← In R, there is no concept of negative indexing.
 $l = [11, 12, 13, 14]$
 Python: 0 1 2 3 /
 -4 -3 -2 -1

Assigning variable in R:

$x \leftarrow 2$ or $2 \rightarrow x$ assigns value "2" to variable x .
 ($x \rightarrow 2$ or $x > 2$ etc. are wrong).

In: $x \leftarrow 2$ class(x) Out: "numeric"	In: $x \leftarrow 2.33$ class(x) Out: "numeric"	In: $x \leftarrow 2L$ class(x) Out: "integer"
--	---	---

"L" indicates to R that it is an integer.

In: $x \leftarrow 2$ class(x) → "characters"
 $x \leftarrow 2 + 3i$ class(x) → "complex"

In: $x \leftarrow 49$
 $x/2$ → 24.5

In: $x \leftarrow 49$
 $x \% / \% 2$
 Out: 24 → prints quotient.

In: $x \% \% 2$
 Out: 1 → prints remainder.

* In R, mathematical operations like $<$, $>$, $<=$, $>=$, $=$, $!=$ are same as python

In: 2^3 → 8
 In: $x = 4$
 $x \% \% 2 == 0$ → TRUE
 } checks whether remainder of $x/2$ is 0 or not, i.e. if x is even.

$A \leftarrow c(4, 5, 6)$: Numeric vector
 $A \leftarrow c("hi", "hello", "bye")$: Character vector.
 $A \leftarrow c(T, F, F, T)$: Logical vector.
 In: $A \leftarrow 1:10$
 A
 Out: 1 2 3 4 ... 10

Letters [1:10] → "a" "b" ... "j"
 LETTERS [1:10] → "A" "B" ... "J"

Letters [c(1,10)] → "a" "j"
 letters [c(1:10)] → "a" "b" ... "j"

Class - 7

In: mixed_vector ← c(1, "hello", TRUE) ← This is a vector, not a list.
 mixed-vector
 Out: "1", "hello", "TRUE" (all become characters).

* vect[-3] → Output: All elements except the 3rd element of vect.
 vect[-2:-4] → Out: All elements except 2nd, 3rd, 4th elements.
 vect[-2, -4] → Out: All elements except 2nd & 4th elements.
 vect[vect > 5] → Out: All elements in vect. that are > 5.

$v_1 \leftarrow c(1, 2, 3)$
 $v_2 \leftarrow c(4, 5, 6)$

$v_1 + v_2$ → 1+4 2+5 3+6 i.e., 5 7 9.
 $v_1 * 2$ → 2 4 6
 $\text{sqrt}(v_1)$ → Square root of each element of v_1 .

* If vectors have different length, the vector having smaller length will be repeated for operations b/w the vectors.

eg. $v_3 \leftarrow c(1, 2)$
 $v_4 \leftarrow c(3, 4, 5, 6, 7)$

$v_3 + v_4$	$\xrightarrow{\text{out}}$	1+3	2+4	1+5	2+6	1+7
		ie., 4	6	6	8	8

Generating sequences: $1:5 \xrightarrow{\text{out}} 1 \ 2 \ 3 \ 4 \ 5$
 $\text{seq}(1, 10, \text{by} = 2) \xrightarrow{\text{out}} 1 \ 3 \ 5 \ 7 \ 9$
 $\text{seq}(5, \text{times} = 3) \xrightarrow{\text{out}} 5 \ 5 \ 5$

$\text{rep}(c("a", "b"), \text{each} = 3) \xrightarrow{\text{out}} \text{aaabbb}$
 $\text{rep}(c("a", "b"), \text{times} = 3) \xrightarrow{\text{out}} \text{ababab}$

In: $\text{my_list} \leftarrow \text{list}("R", 123, \text{TRUE}, c(10, 20, 30), \text{list}("nested", 5))$
 In: $\text{my_list}[[1]] \xrightarrow{\text{out}} "R"$ list within a list.

Similarly, for $[[2]] \rightarrow 123$; $[[3]] \rightarrow \text{TRUE}$; $[[4]] \rightarrow 10 \ 20 \ 30$.
 In: $\text{my_list}[[5]][[1]] \xrightarrow{\text{out}} "nested"$.
 In: $\text{my_list}[[5]][[2]] \xrightarrow{\text{out}} 5$.

} Sub-indexing continues one after another from left to right.

In: $x \leftarrow \text{list}(1, "Hello", \text{list}(2, \text{list}(3, "Hello", \text{list}(1, "Hello"))), \text{list}(1, \text{list}(2, "Hello", \text{list}(1, "Hello"))))$. — Print all "Hello"s.

In: $x[[2]] \xrightarrow{\text{out}} "Hello"$.

In: $x[[3]][[2]][[2]] \xrightarrow{\text{out}} "Hello"$.

In: $x[[3]][[2]][[3]][[2]] \xrightarrow{\text{out}} "Hello"$.

In: $x[[4]][[2]][[2]] \xrightarrow{\text{out}} "Hello"$.

In: $x[[4]][[2]][[3]][[2]] \xrightarrow{\text{out}} "Hello"$.

* We can also write lists in organized manner (multiple lines):

e.g., In: $x \leftarrow \text{list}(1, +2, +3)$.

* Use Shift + Enter to change line of code.

In: $\text{named_list} \leftarrow \text{list}(\text{name} = "Alice", \text{age} = 30, \text{hobbies} = c("reading", "hiking"))$

In: $\text{named_list} \$ \text{name} \xrightarrow{\text{out}} "Alice"$.

$\text{named_list}[[\text{age}]] \xrightarrow{\text{out}}$ Error: object 'age' not found.

$\text{named_list}[[\text{"age"}]] \xrightarrow{\text{out}} 30$.

$\text{named_list}[[\text{"hobbies"}]][[2]] \xrightarrow{\text{out}} "hiking"$.

Matrices in R: In: $\text{my_matrix} \leftarrow \text{matrix}(c(1, 2, 3, 4, 5, 6), \text{nrow} = 2, \text{ncol} = 3)$

In: $\text{my_matrix} \xrightarrow{\text{out}}$

1	3	5
2	4	6

In: $\text{my_matrix}[1,] \xrightarrow{\text{out}} 1 \ 3 \ 5$ (prints 1st row)

In: $\text{my_matrix}[,3] \xrightarrow{\text{out}} 5 \ 6$ (prints 3rd column)

In: $\text{my_matrix}[2,3] \xrightarrow{\text{out}} 6$ (prints the element at A₂₃ position)

** By default, R fills matrices column-wise. If we want to do row-wise allocation instead, do the following:

In: $m2 \leftarrow \text{matrix}(c(1, 2, 3, 4, 5, 6), \text{nrow} = 2, \text{ncol} = 3, \text{byrow} = \text{TRUE})$

Out: $m2$

1	2	3
4	5	6

$\text{byrow} = \text{FALSE}$ will again give default column-wise allocation.